

EnLang Core Language Reference

Book 1: Master Reference & Architecture Guide

Part 1: Introduction to EnLang

Welcome to **Book 1: EnLang Core Language Reference**.

EnLang is the Universal Natural English Programming Language created and authored by **Spandan Prayas Patra**. It allows you to build full-stack web applications, backend algorithms, and database systems using clean, human-readable natural English.

What is EnLang?

EnLang is a deterministic, universal multi-target programming language and transpilation engine that converts natural English into 1:1 clean, production-grade native code targets:

File Extension	Purpose	Native Target Language
.enlg	Core Backend Logic & Algorithms	Python 3
.enlgf	Structural Frontend Markup	HTML5
.enlgd	Styling & Design Systems	CSS3
.enlgs	Client Scripting & DOM Logic	JavaScript (ES6+)
.enlgdb	Database Schemas & Queries	ANSI SQL / SQLite

Why EnLang?

- Human-Readable Natural English:** Write code that reads like plain English sentences (e.g. `define text greeting as "Hello, World!", display greeting`).
- Zero-Config Multi-Target Transpilation:** Seamlessly transpile natural English statements directly to Python, HTML5, CSS3, JavaScript ES6+, and SQL.
- Integrated EPM Package Manager:** Effortlessly install Python dependencies (`epm add py:requests`) and Web libraries (`epm add web:chart.js`) using `epm`.
- Zero Error Compromise:** Built-in static analysis (`enlang check`) and step-by-step interactive debugging (`enlang debug`) to ensure robust execution without unexpected crashes.

Who is this book for?

- **All EnLang Developers:** From beginners learning their first programming language using natural English, to full-stack engineers building production web applications.

- **Language & Compiler Enthusiasts:** Developers looking to understand how natural English syntax transforms 1:1 into high-performance Python and native code.

The Philosophy of EnLang

EnLang prioritizes clarity, performance, and developer happiness. Instead of memorizing obscure syntax rules, curly braces, or complex boilerplate, you state your intent clearly in English, and EnLang handles the rest.

Part 2: Official Installation & Setup

Installing EnLang is fast and straightforward across all platforms (Windows, macOS, Linux). EnLang is distributed globally as a Python package via PyPI.

Official Installation (`pip`)

To install EnLang globally on your system, open your command prompt or terminal and run:

```
pip install enlang
```

Upgrading EnLang

To update to the latest stable release of EnLang at any time, run:

```
enlang update
```

Or via pip:

```
pip install --upgrade enlang
```

Verifying the Installation

After installing, verify that the `enlang` compiler and the `epm` package manager are accessible from your CLI:

```
enlang version
```

Expected Output:

```
EnLang Compiler v1.0.0 (Stable Release)
Authored by Spandan Prayas Patra
```

To list all available versions published on PyPI:

```
enlang versions
```

To switch or install a specific published version:

```
enlang install 1.0.0
```

GUI Installer for Windows

For Windows users who prefer a standalone setup without manually using `pip`, EnLang provides an official executable GUI installer:

- `EnLangInstaller.exe`: Automatically sets up Python dependencies, registers system PATH environment variables, and configures CLI tools (`enlang` and `epm`).
-

Part 3: EnLang Transpiler & Runtime Architecture

Unlike traditional compilers that produce complex binary machine code directly, EnLang uses a **Universal Multi-Target Transpilation Engine**.

Transpiler Architecture

EnLang takes natural English source code (`.enlg`, `.enlgf`, `.enlgd`, `.enlgs`, `.enlgdb`) and performs deterministic 1:1 transpilation into clean, production-grade target code:

- `.enlg` (**Backend Logic**) transpiles to **Python 3**
- `.enlgf` (**Frontend UI**) transpiles to **HTML5**
- `.enlgd` (**Design Systems**) transpiles to **CSS3**
- `.enlgs` (**Client Scripting**) transpiles to **JavaScript (ES6+)**
- `.enlgdb` (**Database Systems**) transpiles to **ANSI SQL / SQLite**

Execution vs Compilation Commands

EnLang provides commands for both direct execution and standalone target compilation:

1. Direct Execution (``enlang run``)

```
enlang run app.enlg
```

Transpiles `app.enlg` in memory to Python 3 and executes it immediately via the built-in runtime engine.

2. Standalone Target Compilation (``enlang build``)

```
enlang build app.enlg
```

Transpiles `app.enlg` and saves the compiled target output to `app.py`.

```
enlang build index.enlgf
```

Compiles `.enlgf` natural markup into clean W3C-compliant `index.html`.

Inline Native Code Blocks

EnLang allows seamless embedding of raw native code within natural English scripts using native block markers:

```
python:
    import math
    print(math.sqrt(16))
end python
```

This guarantees 100% interoperability with native target ecosystems!

Part 4: Complete CLI & EPM Reference + IDE Setup

This part details the official CLI commands for `enlang` and `epm` (EnLang Package Manager), as well as setting up the official EnLang VS Code extension (`vscode-enlang`).

EnLang CLI Command Reference

Execution Commands

```
enlang run app.enlg           # Runs backend Python logic script
enlang run index.enlgf        # Compiles web app & starts live dev server
enlang run index.enlgf -p 3000 # Runs web server on port 3000
enlang run schema.enlgdb      # Compiles SQL, initializes SQLite database & displays tables
enlang server --port 8000     # Starts zero-config HTTP web server
```

Build Commands

```
enlang build main.enlg      # Compiles to main.py
enlang build index.enlgf    # Compiles to index.html
enlang build style.enlgd    # Compiles to style.css
enlang build app.enlgs      # Compiles to app.js
enlang build schema.enlgdb  # Compiles to schema.sql
```

Static Analysis & Debugging

```
enlang check main.enlg      # Runs static linting & syntax analysis
enlang debug main.enlg      # Launches interactive step-by-step debugger
```

EPM Package Manager (`epm`)

The EnLang Package Manager (epm) handles project dependencies:

```
epm init                    # Initializes enlang.json project manifest file
epm add py:requests        # Installs PyPI Python library
epm add web:chart.js        # Installs Web NPM library
epm install                 # Installs all packages declared in enlang.json
```

IDE Setup (VS Code)

The codebase includes an official VS Code extension in `vscode-enlang`:

- Features **Syntax Highlighting** for `.enlg`, `.enlgf`, `.enlgd`, `.enlgs`, and `.enlgdb`.
- Features auto-completion and error diagnostic hints.

Part 5: Language Basics & Natural English Syntax

EnLang is designed so that reading your code feels like reading natural English. This part covers the basic building blocks of an EnLang program (`.enlg`).

Core Syntax Principles

1. **Natural English Keywords:** Instead of cryptic symbols like `{}` or `;`, EnLang uses plain English words (`define`, `as`, `display`, `if`, `then`, `match`, `function`).
2. **Indentation-Based Scoping:** EnLang uses clean Python-like indentation to define blocks of code.
3. **Comments:** Use `#` to write single-line comments in your code.
4. **Case Insensitivity for Sugar:** Keywords like `DEFINE`, `Define`, and `define` are handled gracefully by the transpiler.

Your First EnLang Program (`hello.enlg`)

Create a file named `hello.enlg`:

```
# EnLang Hello World Program
# File: hello.enlg

define text greeting as "Hello, World!"

display greeting
display "Welcome to EnLang Natural English Programming!"
```

Running the Program

Run it directly using the CLI:

```
enlang run hello.enlg
```

Output:

```
Hello, World!  
Welcome to EnLang Natural English Programming!
```

How It Transpiles Under the Hood

When you run `enlang run hello.enlg` or `enlang build hello.enlg`, the transpiler translates it into native Python:

```
# Transpiled from hello.enlg  
greeting = "Hello, World!"  
print(greeting)  
print("Welcome to EnLang Natural English Programming!")
```

Statement Separators & Line Rules

- Every statement in EnLang is placed on a new line.
 - You do not need semicolons ; at the end of lines.
 - Indentation must be consistent (4 spaces per indentation level is recommended).
-

Part 6: Variables & Value Assignment

Variables in EnLang allow you to store and manipulate data using plain English syntax.

Variable Declaration Syntax

You can declare typed variables using `define`, `let`, or `var`:

Format:

```
define <type> <name> as <value>  
# OR  
set <name> to <value>  
# OR  
store <value> in <name>
```

Examples:

```
define text user_name as "Spandan"  
define number user_age as 25  
define decimal account_balance as 1500.50  
define boolean is_verified as true
```

Default Initializations

If you declare a typed variable without specifying an initial value, EnLang automatically initializes it to a safe default:

```
define number score          # Defaults to 0  
define decimal rate          # Defaults to 0.0  
define text title            # Defaults to ""  
define boolean active        # Defaults to false  
define list items            # Defaults to []  
define dictionary config     # Defaults to {}
```

Reassigning Values

You can update the value of an existing variable using natural English phrasing:

```
set score to 100  
store "Spandan Prayas Patra" in user_name  
set is_verified to false
```

Transpilation Mapping

```
| EnLang Natural Syntax | Transpiled Native Python Target |  
| :--- | :--- |  
| define text name as "Spandan" | name = "Spandan" |  
| define number count as 10 | count = 10 |  
| set count to 20 | count = 20 |  
| store 50 in count | count = 50 |  
| define list users | users = [] |  
| define dictionary settings | settings = {} |
```

Part 7: Primitive & Collection Data Types

EnLang supports a rich set of built-in primitive and collection data types.

1. Text (`text`)

Represents string literals.

```
define text title as "EnLang Documentation"  
define text message as 'Natural English Syntax'
```

2. Numbers (`number` & `decimal`)

- **number**: Whole integers.
- **decimal**: Floating-point numbers.

```
define number total_items as 42  
define decimal temperature as 98.6
```

3. Booleans (`boolean`)

Represents true or false truth values.

```
define boolean is_logged_in as true  
define boolean has_permission as false
```

4. Lists (`list` / `array`)

Ordered collections of items.

```
define list fruits as ["Apple", "Banana", "Cherry"]  
define list numbers as [10, 20, 30, 40]
```

5. Dictionaries (`dictionary` / `dict` / `map`)

Key-value mappings.

```
define dictionary user as {"name": "Spandan", "role": "Author"}
```

6. Sets (`set`)

Unordered collections of unique elements.

```
define set unique_ids
```

Summary Table

	Data Type	Keyword	Natural Declaration	Example	Default Value
		:	---	:	---
String	text	define	text	city as "Delhi"	""
Integer	number	define	number	age as 25	0
Float	decimal	define	decimal	price as 99.99	0.0
Boolean	boolean	define	boolean	is_admin as true	False
List	list/array	define	list	tags as ["ai", "web"]	[]
Dictionary	dictionary/map	define	dictionary	config	{}
Set	set	define	set	items	set()

Part 8: Operators & Expression Cleaners

In EnLang, expressions use natural English operators instead of standard programming symbols.

Comparison Operators

	EnLang	Natural Operator	Native Python Equivalent	Example
	:	---	:	---
is equal to	==	if score is equal to 100 then:		
is not equal to	!=	if status is not equal to "active" then:		
is greater than	>	if age is greater than 18 then:		
is less than	<	if price is less than 50 then:		
is greater than or equal to	>=	if score is greater than or equal to 80 then:		
is less than or equal to	<=	if count is less than or equal to 5 then:		
is in	in	if "admin" is in roles then:		
is not in	not in	if item is not in cart then:		

Arithmetic Operators

	EnLang	Natural Operator	Native Python Equivalent	Example
	:	---	:	---
plus	+	set total to price plus tax		
minus	-	set balance to total minus discount		
times	*	set area to width times height		
divided by	/	set average to sum divided by count		
modulo	%	set remainder to number modulo 2		
power of	**	set result to 2 power of 8		

Logical Operators

| EnLang Natural Operator | Native Python Equivalent | Example |

| :--- | :--- | :--- |

| and | and | if age > 18 and is_active is true then: |

| or | or | if role == "admin" or role == "editor" then: |

| not | not | if not is_disabled then: |

Natural Expression Examples

```
define number a as 10
define number b as 20
```

```
if a plus b is equal to 30 then:
    display "Math is correct!"
```

Transpiles directly to:

```
a = 10
b = 20
if a + b == 30:
    print("Math is correct!")
```

Part 9: Control Flow, Conditional Logic & Pattern Matching

Control flow structures dictate the order in which statements are executed in an EnLang program.

1. Conditional Logic (`if` / `else`)

EnLang uses natural `if ... then:` statements.

Syntax:

```
if <condition> then:
    <statements>
otherwise if <condition> then:
    <statements>
otherwise:
    <statements>
```

Real Example:

```
define number user_age as 20

if user_age is greater than 18 then:
    display "Access Granted: Adult"
otherwise if user_age is equal to 18 then:
    display "Access Granted: Newly Adult"
otherwise:
    display "Access Denied: Minor"
```

Native Python Target Output:

```
user_age = 20
if user_age > 18:
    print("Access Granted: Adult")
elif user_age == 18:
    print("Access Granted: Newly Adult")
else:
    print("Access Denied: Minor")
```

2. Pattern Matching (`match` / `case`)

EnLang features a powerful pattern matching syntax (match, case, default, end match):

```
define text status_code as "200"

match status_code:
case "200":
    display "Success OK"
case "404":
    display "Error: Resource Not Found"
case "500":
    display "Error: Internal Server Error"
default:
    display "Unknown Status Code"
end match
```

Match with Multiple Values & Expressions:

```
define number score as 85

match score:
case is greater than or equal to 90:
    display "Grade A"
case is greater than or equal to 80:
    display "Grade B"
default:
    display "Grade C"
end match
```

3. Increment & Decrement Shortcuts

EnLang provides natural English syntax for updating numerical variables:

```
define number score as 10
increment score by 5      # Transpiles to: score += 5
decrement score by 2      # Transpiles to: score -= 2
```

Part 10: Functions & Async Operations

Functions encapsulate reusable logic in EnLang.

1. Defining & Calling Functions

Functions are defined using the function keyword:

```
function calculate_total(price, tax_rate):
    define decimal tax as price times tax_rate
    return price plus tax

define decimal final_price as calculate_total(100.0, 0.18)
display "Final Price: " + final_price
```

Transpiled Target Output:

```
def calculate_total(price, tax_rate):
    tax = price * tax_rate
    return price + tax

final_price = calculate_total(100.0, 0.18)
print("Final Price: " + str(final_price))
```

2. Asynchronous Functions (`async`)

EnLang natively supports asynchronous functions for non-blocking operations:

```
async function fetch_user_data(user_id):
    display "Fetching data asynchronously for user: " + user_id
    fetch url "https://api.example.com/users/" + user_id and store in response
    return response
```

Transpiled Target Output:

```
async def fetch_user_data(user_id):
    print("Fetching data asynchronously for user: " + str(user_id))
    import urllib.request
    response = urllib.request.urlopen("https://api.example.com/users/" + str(user_id)).read().decode('utf-8')
    return response
```

3. Built-in Utility Functions

EnLang features built-in natural functions for math, strings, and system delays:

```
sleep 2 seconds           # Pauses execution for 2 seconds
sleep 500 ms              # Pauses execution for 500 milliseconds
```

```
get current date and time and store in current_now
display "Current time: " + current_now
```

Part 11: Object-Oriented Programming (OOP) & Interfaces

EnLang supports full Object-Oriented Programming, including Classes, Inheritance, and Interfaces.

1. Creating Interfaces (`interface`)

Interfaces define blueprints for classes:

```
create interface Authenticatable:
    function login(username, password):
    function logout():
end interface
```

Transpiled Target Output:

```
class Authenticatable:
    pass
# end class/interface
```

2. Classes & Inheritance (`create class` / `extends` / `implements`)

Classes are created using `create class` with optional `extends` (for inheritance) and `implements`:

```
create class BaseUser:
    function get_role():
        return "Standard User"

create class User extends BaseUser:
    function __init__(self, username, email):
        set self.username to username
        set self.email to email

    function get_info(self):
        return self.username + " (" + self.email + ")"
```

Transpiled Target Output:

```
class BaseUser:
    def get_role(self):
        return "Standard User"

class User(BaseUser):
    def __init__(self, username, email):
        self.username = username
        self.email = email

    def get_info(self):
        return self.username + " (" + str(self.email) + ")"
```

3. Instantiating Objects

Objects are instantiated naturally:

```
define user1 as User("Spandan", "spandan@enlang.org")
display user1.get_info()
```

Part 12: Error Handling & Exception Management

Robust applications must handle errors gracefully without unexpected crashes. EnLang provides natural English keywords for throwing, catching, and handling exceptions.

1. Raising Exceptions (`raise` / `throw`)

You can throw exceptions using natural syntax:

```
define number age as -5

if age is less than 0 then:
    raise ValueError with message "Age cannot be negative"
```

Or using throw error:

```
if connection_failed is true then:
    throw error "Database Connection Timeout"
```

Transpiled Target Output:

```
if age < 0:
    raise ValueError("Age cannot be negative")

if connection_failed == True:
    raise Exception("Database Connection Timeout")
```

2. Catching Exceptions (`try` / `except` / `finally`)

EnLang supports standard try-except exception blocks:

```
try:
    read file "data.txt" into content
    display content
except FileNotFoundError:
    display "Warning: data.txt was not found!"
finally:
    display "Cleanup completed."
```

Transpiled Target Output:

```
try:
    with open("data.txt", 'r', encoding='utf-8') as _f: content = _f.read()
    print(content)
except FileNotFoundError:
    print("Warning: data.txt was not found!")
finally:
    print("Cleanup completed.")
```

Zero Error Philosophy

EnLang emphasizes catching errors at static compile time using `enlang check main.enlg`. This prevents cryptic runtime 500 server crashes in production!

Part 13: Modules, Packages & File Linking

Organizing code into modular files and importing third-party libraries is essential for large-scale applications.

1. Importing Modules (`import module`)

To import external or standard library modules in EnLang:

```
import module math as m
import module datetime

define number root as m.sqrt(16)
display "Square root of 16 is: " + root
```

Transpiled Target Output:

```
import math as m
import datetime

root = m.sqrt(16)
print("Square root of 16 is: " + str(root))
```

2. Selective Imports (`from ... import`)

You can import specific functions or classes from a module:

```
from math import sqrt, floor

define number result as floor(sqrt(20))
display "Floor of sqrt(20): " + result
```

3. Linking & Including Other EnLang Files (`include`)

EnLang allows linking other `.enlg`, `.enlgf`, `.enlgd`, `.enlgs`, or `.enlgdb` files dynamically:

```
include "helper.enlg"
include "style.enlgd"
```

When transpiled, EnLang reads and executes the referenced target file within the current scope seamlessly.

Part 14: File I/O & Storage Operations

EnLang provides natural English syntax for reading from and writing to disk files.

1. Reading Files (`read file`)

To read the full contents of a file into a variable:

```
read file "config.json" into config_data
display "Config loaded: " + config_data
```

Transpiled Target Output:

```
with open("config.json", 'r', encoding='utf-8') as _f:
    config_data = _f.read()
print("Config loaded: " + str(config_data))
```

2. Writing to Files (`write ... to file`)

To write text or variable data to a file:

```
define text log_entry as "User logged in at 10:00 AM"
write log_entry to file "app.log"
```

Transpiled Target Output:

```
log_entry = "User logged in at 10:00 AM"
with open("app.log", 'w', encoding='utf-8') as _f:
    _f.write(str(log_entry))
```

Part 15: Web Server, Cryptography & HTTP Networking

EnLang includes built-in commands for starting web servers, making HTTP network requests, and performing cryptographic hashing.

1. Zero-Config HTTP Web Server

You can start a lightweight web server directly from `.enlg` code:

```
start web server on port 8000
```

Transpiled Target Output:

```
from enlang_core.web_server import start_enlang_server
start_enlang_server(8000)
```

2. HTTP Networking (`fetch url`)

Fetch external data over HTTP/HTTPS:

```
fetch url "https://api.github.com/users/spandan" and store in response
display response
```

Transpiled Target Output:

```
import urllib.request
response = urllib.request.urlopen("https://api.github.com/users/spandan").read().decode('utf-8')
print(response)
```

3. Cryptographic Hashing (`hash`)

Perform secure cryptographic hashing (SHA256, MD5, SHA512) natively:

```
define text secret as "MyPassword123"
hash secret with sha256 and store in hashed_password

display "Hashed Password: " + hashed_password
```

Transpiled Target Output:

```
secret = "MyPassword123"
import hashlib
hashed_password = hashlib.sha256(secret.encode('utf-8')).hexdigest()
print("Hashed Password: " + str(hashed_password))
```

Part 16: Built-in Natural Language Processing (NLP) Engine

One of EnLang's most unique capabilities is built-in Natural Language Processing (NLP). You can perform sentiment analysis, keyword extraction, and text similarity calculations using plain English syntax.

1. Sentiment Analysis (`analyze sentiment`)

Analyze whether a body of text is positive, negative, or neutral:

```
define text review as "EnLang is an amazingly intuitive and fast language!"

analyze sentiment of review and store in sentiment_score
display "Sentiment: " + sentiment_score
```

Transpiled Target Output:

```
review = "EnLang is an amazingly intuitive and fast language!"
from enlang_core.nlp_engine import analyze_sentiment
sentiment_score = analyze_sentiment(review)
print("Sentiment: " + str(sentiment_score))
```

2. Keyword Extraction (`extract keywords`)

Automatically extract key terms from unstructured text:

```
define text article as "Artificial intelligence and compiler design are advancing rapidly in 2026."

extract keywords from article into keywords_list
display "Keywords: " + keywords_list
```

Transpiled Target Output:

```
article = "Artificial intelligence and compiler design are advancing rapidly in 2026."
from enlang_core.nlp_engine import extract_keywords
keywords_list = extract_keywords(article)
print("Keywords: " + str(keywords_list))
```

3. Text Similarity Calculation (`calculate similarity`)

Compare two text strings and calculate their similarity score:

```
define text text1 as "Build full-stack web applications in English"
define text text2 as "Create web apps using natural English code"

calculate similarity between text1 and text2 and store in score
display "Similarity Score: " + score
```

Transpiled Target Output:

```
text1 = "Build full-stack web applications in English"
text2 = "Create web apps using natural English code"
from enlang_core.nlp_engine import calculate_similarity
score = calculate_similarity(text1, text2)
print("Similarity Score: " + str(score))
```

Part 17: Database Integration (`.enlg` & `.enlgdb`)

EnLang provides built-in database support directly from `.enlg` scripts, as well as standalone `.enlgdb` schema files.

1. Connecting to SQLite Database (`.connect to database`)

```
connect to database "app.db" as db
```

Transpiled Target Output:

```
import sqlite3
db = sqlite3.connect("app.db")
```

2. Defining Tables (`.define table`)

Define database tables using natural English column declarations:

```
define table users with columns id as INTEGER PRIMARY KEY AUTOINCREMENT, username as TEXT NOT NULL, email as TEXT NOT NULL
```

Transpiled Target Output:

```
_cur = db.cursor()
_cur.execute('CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT NOT NULL, email TEXT NOT NULL)')
db.commit()
```

3. Inserting Records (`.insert record`)

```
insert record into users with values NULL, 'Spandan', 'spandan@enlang.org'
```

Transpiled Target Output:

```
_cur = db.cursor()
_cur.execute(f'INSERT INTO users VALUES (NULL, \'Spandan\', \'spandan@enlang.org\')')
db.commit()
```

4. Executing Custom SQL Queries (`.execute query`)

Execute queries and store results directly into EnLang variables:

```
execute query "SELECT * FROM users" on database db and store in all_users
```

```
display all_users
```

Transpiled Target Output:

```
_cur = db.cursor()
_cur.execute("SELECT * FROM users")
all_users = _cur.fetchall()
db.commit()
print(all_users)
```

Part 18: Testing, Interactive Debugging & Static Analysis

To ensure production stability, EnLang features integrated static linting, interactive step-by-step debugging, and testing utilities.

1. Static Linting & Analysis (`.enlang check`)

Run `enlang check` before running your code to catch syntax and logic errors early:

```
enlang check main.enlg
```

Output:

```
[CHECK] Analyzing main.enlg...
[OK] Syntax check passed. 0 Errors, 0 Warnings found.
```

If an error exists, `enlang` check provides detailed line numbers and exact suggestions for fixing it.

2. Interactive Debugger (`enlang debug`)

Launch the interactive step-by-step debugger to inspect variable values during execution:

```
enlang debug main.enlg
```

Interactive Debug Commands:

- `step / s`: Execute the next line of EnLang code.
- `print / p`: Inspect current runtime value of .
- `continue / c`: Resume execution until the next breakpoint.
- `quit / q`: Exit the debugging session.

3. Integrated Test Runner (`enlang test`)

Write automated test functions starting with `test_`:

```
function test_addition():
  define number a as 5
  define number b as 10
  assert a plus b is equal to 15
```

Run all tests in your workspace:

```
enlang test
```

Output:

```
[TEST] Running test suite...
✓ test_addition PASSED (0.002s)
[SUMMARY] 1 passed, 0 failed.
```

Part 19: Complete Language Keyword Matrix & Specification

This reference chapter summarizes the core keywords, file extensions, and target transpilation mappings of the EnLang language suite.

File Extension Matrix

Extension	Domain	Target Language	CLI Compile Command
-----------	--------	-----------------	---------------------

----	----	----	----
------	------	------	------

.enlg	Core Backend Logic & Algorithms	Python 3	enlang build main.enlg
-------	---------------------------------	----------	------------------------

.enlgf	Structural Frontend Markup	HTML5	enlang build index.enlgf
--------	----------------------------	-------	--------------------------

.enlgd	Styling & Design Systems	CSS3	enlang build style.enlgd
--------	--------------------------	------	--------------------------

.enlgs	Client Scripting & DOM Logic	JavaScript (ES6+)	enlang build app.enlgs
--------	------------------------------	-------------------	------------------------

.enlgdb	Database Schemas & Queries	ANSI SQL / SQLite	enlang build schema.enlgdb
---------	----------------------------	-------------------	----------------------------

Keyword Transpilation Matrix

Natural English Syntax	Domain	Transpiled Output
------------------------	--------	-------------------

```

|:---|:---|:---|
|define as |Logic|= |
|set to |Logic|= |
|store in |Logic|= |
|display |Logic|print()|
|ask and store in |Logic|= input()|
|if then:|Logic|if :|
|otherwise if then:|Logic|elif :|
|otherwise:|Logic|else:|
|match :|Logic|Pattern matching construct|
|function (:)|Logic|def (:)|
|async function (:)|Logic|async def (:)|
|create class extends :|Logic|class (:)|
|create interface :|Logic|class :|
|raise with message |Logic|raise ()|
|read file into |File I/O|with open(...) as _f: = _f.read()|
|write to file |File I/O|with open(...) as _f: _f.write()|
|connect to database as |DB|sqlite3.connect()|
|define table with columns ...|DB|CREATE TABLE (...)|
|start web server on port |Server|start_enlang_server()|
|hash with and store in |Security|hashlib(...).hexdigest()|
|fetch url and store in |Network|urllib.request.urlopen(...)|
|analyze sentiment of and store in |NLP|analyze_sentiment()|
|extract keywords from into |NLP|extract_keywords()|
|calculate similarity between and |NLP|calculate_similarity(, )|

```